Quant Project

Session Summary

Date: May 13, 2026

1. What Was Built This Session

Multi-Strategy Architecture Restructure

The project was reorganised from a flat single-strategy layout into a scalable multi-strategy architecture. Every future strategy now has a clearly defined home.

- src/strategies/pairs/ -all pairs-trading logic (spread, signals, config, viz, backtest)
- src/analytics/ -shared statistical tools (ADF test, half-life)
- src/backtest/ -strategy-agnostic backtest engine and metrics
- src/viz/theme.py -shared color palette imported by all strategy charts
- notebooks/pairs/ -notebooks moved into a strategy subfolder

Adding a future strategy (momentum, mean reversion, options) now requires exactly two things: a new src/strategies/<name>/ folder and one registration function in run.py.

Data Layer Upgrade

The data fetcher previously cached only closing prices. It now fetches and caches full OHLCV data (Open, High, Low, Close, Volume) for every ticker. Old single-column cache files are automatically refreshed on next use.

- fetch_ohlcv(ticker, period) -single ticker, returns 5-column DataFrame
- fetch_pair_ohlcv(ticker_a, ticker_b) -aligned OHLCV for both legs of a pair
- fetch_price / fetch_pair -unchanged signatures; backward compatible

Backtest Engine

A full vectorised backtest engine was built. Given a signal DataFrame from the pairs pipeline, it simulates every trade, builds an equity curve, and computes performance statistics.

- src/backtest/engine.py -run_backtest(): trade log -> daily equity curve
- src/backtest/metrics.py -compute_metrics(): equity curve -> performance dict
- src/strategies/pairs/backtest.py -build_trade_log() with hedge-ratio-adjusted sizing

Position sizing is dollar-neutral and hedge-ratio adjusted per leg ($10,000 default). P&L uses close-to-close fills, a known simplification suitable for daily bars at this stage.

New Output Windows (--backtest flag)

Running with --backtest adds four new browser windows to the existing three:

- Equity Curve -two-panel: portfolio value + drawdown, shaded trade bands
- Trade P&L -bar chart of per-trade returns (green = win, red = loss)
- Backtest Metrics -styled table: returns, Sharpe, drawdown, win rate, profit factor
- Backtest Explanation -contextual plain-English interpretation of every metric

Named Browser Tabs

All browser windows now open with descriptive tab titles instead of numbers. The _show(fig, title) helper in run.py injects a <title> tag into each HTML export.

- Examples: 'KO/PEP -Stats', 'KO/PEP -Charts', 'KO/PEP -Backtest Explanation'

2. How to Run

Signal analysis only (3 windows)

```
python run.py pairs --pair KO/PEP
```

Signal analysis + backtest (7 windows)

```
python run.py pairs --pair KO/PEP --backtest
```

Scan all configured pairs

```
python run.py pairs --all
```

Custom parameters

```
python run.py pairs --pair JPM/BAC --period 1y --window 30 --z-entry 1.8 --backtest
```

Help

```
python run.py --help
python run.py pairs --help
```

3. Current Project Structure

File	Purpose
`run.py`	CLI entry point -subcommand-based
`requirements.txt`	Python dependencies
`data/cache/`	Auto-populated OHLCV CSV cache
`docs/`	Session summaries and documentation
`notebooks/pairs/01_pairs_trading.ipynb`	Deep-dive: cointegration, spread, signals
`notebooks/pairs/02_scenarios.ipynb`	Sandbox: parameter tuning, custom pairs
`src/data/fetcher.py`	yfinance wrapper -fetch_ohlcv, fetch_pair_ohlcv
`src/analytics/stationarity.py`	adf_test, compute_half_life (shared)
`src/backtest/engine.py`	run_backtest() -trade log to equity curve
`src/backtest/metrics.py`	compute_metrics() -performance statistics
`src/viz/theme.py`	Shared color palette constants
`src/strategies/base.py`	Abstract BaseStrategy interface
`src/strategies/pairs/config.py`	PAIRS list and DEFAULT_PARAMS
`src/strategies/pairs/spread.py`	compute_hedge_ratio, compute_spread
`src/strategies/pairs/signals.py`	compute_zscore, generate_signals
`src/strategies/pairs/backtest.py`	build_trade_log, run_pairs_backtest
`src/strategies/pairs/viz.py`	All Plotly chart functions (10 total)

4. Key Concepts Covered

Cointegration — Two non-stationary series whose linear combination is stationary. The statistical foundation of pairs trading.

OLS Hedge Ratio (b) — Estimated by regressing log(A) on log(B). Makes the spread as stationary as possible. Drives position sizing.

ADF Test — Augmented Dickey-Fuller test. p < 0.05 means the spread is stationary -the core assumption is validated.

Half-Life — From AR(1) fit: how many days for the spread to revert halfway to its mean. Guides rolling window and holding period.

Rolling Z-Score — z = (spread - rolling_mean) / rolling_std. Expresses divergence in standard deviations for threshold-based signals.

State Machine — Signal generator that tracks position state to prevent overlapping trades. Entry / hold / exit or stop.

Vectorised Backtest — Compute all signals first, then replay trades in a single forward pass. Simpler than event-based; fine for daily bars.

Sharpe Ratio — mean(daily returns) / std(daily returns) * sqrt(252). Risk-adjusted return. > 1.0 is good; negative means losing on risk basis.

Profit Factor — Sum of winning P&L / sum of losing P&L. > 1.0 required for profitability. > 1.5 is the professional target.

Max Drawdown	Worst peak-to-trough equity decline. Measures the worst-case experience of holding the strategy.

5. Key Finding: Current Configured Pairs

None of the four currently configured pairs (KO/PEP, JPM/BAC, XOM/CVX, MSFT/GOOGL) passed the ADF stationarity test over the 2-year lookback window. This is a real market finding, not a code bug -the pairs have diverged structurally in recent years.

KO went +32% while PEP fell -10% over the same period (opposite directions), producing a hedge ratio near zero and a non-stationary spread. Before running a live backtest, finding genuinely cointegrated pairs is the most important next step.

6. Future Recommendations

Immediate Priority: Find Valid Pairs

The current pairs fail the stationarity test. Before improving the backtest engine further, find pairs that actually cointegrate. Good candidates to test:

- GS / MS (Goldman Sachs vs Morgan Stanley -investment banking peers)
- DAL / UAL (Delta vs United -same routes, same fuel costs)
- CVS / WBA (pharmacy retail duopoly)
- Try shorter periods (--period 1y or 6mo) on existing pairs -a shorter window may reveal a valid regime

Automated Pair Discovery

Instead of hand-picking pairs, scan a universe of stocks for cointegrated candidates. This is the next major analytical feature:

- Build src/analytics/cointegration.py -Johansen test (tests multiple series simultaneously)
- Add a scan command: python run.py pairs --scan-universe SP500 --sector Financials
- Filter by: ADF p-value, half-life range, beta range, minimum trade count

Rolling Hedge Ratio

Currently beta is estimated using the full lookback window -an in-sample simplification that introduces look-ahead bias. A rolling beta re-estimates it using only past data:

- Add compute_rolling_hedge_ratio(series_a, series_b, window) to spread.py
- Typical window: 252 days (1 year). Recalculate daily or weekly
- This is a prerequisite for a truly out-of-sample backtest

Walk-Forward Validation

The current backtest trains and tests on the same data window -a form of in-sample testing. Walk-forward validation splits the data into sequential train/test folds:

- Train on first N months (estimate beta, confirm stationarity)
- Test on the following M months (run signals and backtest on unseen data)
- Roll forward and repeat. Average performance across all test windows
- This is the gold standard for validating a systematic strategy before trading

Transaction Costs and Slippage

The current backtest assumes zero friction. In reality, even with zero-commission brokers, slippage (the difference between the signal price and the fill price) eats into P&L:

- Add slippage_bps parameter to run_pairs_backtest() -e.g., 5 bps round-trip
- Adjust entry/exit prices: entry_price * (1 + slippage) for buys, * (1 - slippage) for sells
- At 5 bps on $20,000 capital that's ~$10 per round-trip -meaningful on short-hold trades

Position Sizing

Current sizing is fixed at $10,000 per leg regardless of signal strength. Better approaches:

- Size by z-score magnitude: larger position when z-score is more extreme
- Size by volatility: reduce size when the spread is unusually volatile (VaR-based)

- Kelly criterion: size based on estimated edge and variance -academic but powerful

Alpaca Integration (Paper Trading)

When ready to move beyond backtesting, Alpaca (alpaca.markets) is the recommended next data and execution layer. It offers:

- Free paper trading account -simulate real orders with real market data
- Historical OHLCV data API (replaces yfinance -more reliable, official)
- Same Python SDK (alpaca-py) for paper and live trading -minimal code change
- The fetcher.py abstraction is already designed for this swap

Migration path: add AlpacaProvider class that implements the same fetch_ohlcv() interface. Switch by changing one import.

Additional Strategies

The multi-strategy architecture is ready. Next strategies to consider adding (in rough order of complexity):

- Momentum -buy stocks making new highs, short those making new lows (trend following)
- Mean Reversion (single name) -Bollinger Band entries on individual stocks
- ETF Arbitrage -pairs between an ETF and a basket of its components
- Sector rotation -rank sectors by relative momentum, rotate monthly

Notebook Updates

The notebooks haven't been updated to reflect the backtest engine yet. Next notebook additions:

- 03_backtest.ipynb -walkthrough of the backtest mechanics, equity curve interpretation
- Update 01_pairs_trading.ipynb -add a section on why the current pairs are failing the ADF test
- Update 02_scenarios.ipynb -add scenario comparison for different backtest periods